

一.引入网络编程

1.引入

前面学习的进程间通信：只能用于同一台主机内部的进程间通信(不能在不同主机间通信)
网络编程：也是属于进程间通信的一种手段，不仅可以在同一台主机内部通信，也能在不同的主机间通信

2.知识点概览

- 传输层的两个重要协议tcp和udp
 - tcp的通信流程
 - tcp的单向通信
 - tcp的双向通信
 - udp的通信流程
 - udp单向，双向通信
- 多路复用
- http获取天气信息，json数据解析

二.网络有关的基本概念

1.网络模型

OSI七层模型

层级	名称	核心作用	常用协议
7	应用层	为应用程序提供网络服务接口，直接面向用户需求	HTTP/HTTPS、FTP、SMTP/POP3/IMAP、DNS、DHCP、Telnet、SSH
6	表示层	处理数据的格式转换、加密解密、压缩解压缩，确保通信双方能识别数据	JPEG、MP3、TLS/SSL、ASCII、Unicode
5	会话层	建立、管理和终止通信双方的会话连接，控制会话交互方式	RPC、NetBIOS、PPTP、SOCKS
4	传输层	负责端到端的可靠数据传输，处理分段、重传、流量控制	TCP、UDP、SCTP（流控制传输协议）
3	网络层	实现不同网络间的寻址和路由选择，将分组转发到目标网络	IP（IPv4/IPv6）、ICMP、IGMP、ARP/RARP、OSPF、BGP、RIP
2	数据链路层	将物理层的信号封装成帧，处理 MAC 地址寻址和链路级错误检测	Ethernet（以太网）、PPP、HDLC、WIFI MAC、LLC、VLAN（802.1Q）
1	物理层	定义物理介质的电气、机械特性，传输二进制比特流	RJ45、光纤协议、RS-232、IEEE 802.3（物理层部分）、蓝牙物理层

TCP/IP模型

- 把七层模型合并简化成四层
- 应用层(原来的应用层，会话层，表示层合并)
- 传输层

网际层、IP层
网络接口层(数据链路层和物理层合并)

2.ipv4和ipv6

ipv4： 32位的ip地址，用三个小数点把ip划分为四个部分(点分十进制ip)，每个部分各占一个字节(0-255之间)
比如: 192.168.11.1 //小数点不占用存储空间，ip地址在存放的时候只是存放了四个字节的数

ipv6： 解决ipv4地址不够用,128位的ip地址

查看linux的ip

ifconfig

修改linux的ip

sudo ifconfig 虚拟网卡的名字 要修改的ip

3.端口号

作用：区分同一台主机内部不同的网络进程

本质就是个无符号的短整型数(0---65535之间)

程序员编程的时候可以自己指定端口号，但是不能使用1024以内(很多被操作系统或者一些通用的网络协议占用了)

ssh服务22 http 80 https 443

协议名称	端口号	传输层协议	核心用途
HTTP	80	TCP	超文本传输，普通网页访问
HTTPS	443	TCP	加密的 HTTP，安全网页 / 接口通信
FTP	21（控制）/20（数据）	TCP	文件传输协议，控制连接与数据连接分离
SSH	22	TCP	安全远程登录 / 文件传输，替代 Telnet
Telnet	23	TCP	明文远程登录（嵌入式调试常用，安全性低）
SMTP	25	TCP	邮件发送协议
DNS	53	TCP/UDP	域名解析（UDP 用于查询，TCP 用于传输大报文）
DHCP	67（服务器）/68（客户端）	UDP	动态分配 IP 地址，嵌入式设备自动联网常用
TFTP	69	UDP	简单文件传输协议，嵌入式系统烧录固件常用

4.不同主机之间网络编程注意事项

第一个：不同主机之间，网路必须是连通的

使用ping命令，去ping一下对象的ip地址，看看是否有延时出现，有的话说明网络是通的

例如： ping 对方的ip地址

三.tcp通信协议(传输控制协议Transfer Control Protocol)

1.通信流程(通过男女朋友打电话类比)



2.相关的接口函数

(1)创建tcp套接字(买手机)

```
int socket(int domain, int type, int protocol);
```

返回值：成功 返回套接字的文件描述符

失败 -1

参数: domain --》地址协议类型

IPV4地址协议 --》 AF_INET 或者PF_INET

IPV6地址协议 --》 AF_INET6 或者PF_INET6

type --》套接字的类型

tcp套接字(流式套接字，数据流套接字) --》 SOCK_STREAM

udp套接字(数据报套接字) --》 SOCK_DGRAM

protocol --》扩展协议，一般设置0，不用理会

(2)绑定ip和端口号

```
int bind(int sockfd, const struct sockaddr *addr,socklen_t addrlen);
```

返回值：成功 返回0

失败 返回-1

参数: sockfd --》套接字的文件描述符

addr(重点，重点) --》

struct sockaddr 通用地址结构体(兼容ipv4和ipv6)

struct sockaddr_in6 ipv6地址结构体(专门存放ipv6地址的)

struct sockaddr_in ipv4地址结构体(专门存放ipv4地址的)

```
{
```

sin_family; //存放地址协议的 AF_INET 或者PF_INET

struct in_addr sin_addr; //存放需要绑定ipv4地址(绑定都是绑定自己的ip)

sin_port; //存放需要绑定的端口号

```
}
```

```
struct in_addr
```

```
{
    in_addr_t s_addr; //最终存放网络字节序ip地址的变量
}
```

addrlen --》地址结构体的大小 sizeof()

💡 linux中提供了宏定义INADDR_ANY在绑定的时候帮助我们寻找本地主机中任意一个ip地址帮我们绑定

例如: bindaddr.sin_addr.s_addr=inet_addr(具体的ip地址) //以前的写法, 有局限性, 换了一台电脑, 需要修改ip
 bindaddr.sin_addr.s_addr=htonl(INADDR_ANY); //自动匹配本机ubuntu上的ip, 只能在绑定的时候使用

(3)字节序的转换

第一: 概念

计算机网络中数据采用大端序存放的 ---》网络字节序(大端序)

ubuntu系统中数据采用小端序存放的 ---》主机字节序(小端序)

第二: 小端序(主机字节序)--》转成大端序(网络字节序)

应用场景: bind函数绑定本地主机的ip地址和端口号

小端序ip--》大端序ip

```
in_addr_t inet_addr(const char *cp);
```

返回值: 成功返回转换得到的大端序ip

参数: cp --》你传递过来字符串格式的ip地址

小端端口号--》大端端口号

```
uint16_t htons(uint16_t hostshort);
```

返回值: 成功 大端序端口号

参数: hostshort --》小端序端口号

第三: 大端序(网络字节序)--》转成小端序(主机字节序)

应用场景: accept成功之后打印客户端的ip和端口

大端ip--》小端ip

```
char *inet_ntoa(struct in_addr in);
```

返回值: 转换得到的小端序字符串ip

参数: in --》大端序的ip

大端端口号--》小端端口号

```
uint16_t ntohs(uint16_t netshort);
```

(4)连接(拨号)

```
int connect(int socket, const struct sockaddr *address, socklen_t address_len);
```

参数: address(重点) --》存放对方(服务器)的ip和端口号

(5)监听(待机)

```
int listen(int socket, int backlog);
```

参数: backlog(重点) --》同时最多允许多少个客户端连接服务器

例如: listen(tcpsock,5); 表示同时最多允许5个客户端连接服务器

(6)接受客户端的连接请求(愿意接听)

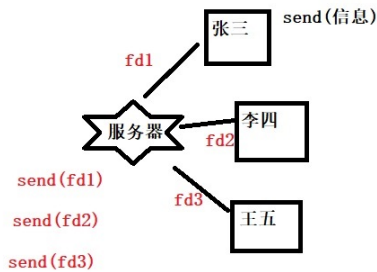
```
int accept(int socket, struct sockaddr *restrict address, socklen_t *restrict address_len);
```

返回值: (重点)

成功返回新的套接字(用于跟对应的客户端通信的) 失败-1

由于tcp允许多个客户端连接同一个服务器, 服务器如何区分不同的客户端呢?

答案: accept给每个连接成功的客户端都产生一个新的套接字, 用这个套接字去区分不同的客户端即可



参数: address(重点) --》存放对方的ip和端口号

当某个客户端连接服务器成功, accept会自动把该客户端的ip和端口号自动存储到该地址中

address_len --》地址的大小, 要求是指针

总结accept的特点:

◆ 如果没有客户端连接服务器, 那么服务器会阻塞在accept这里, 一旦有客户端连接成功, accept立马产生新的套接字

(7)两组常用的tcp收发信息的函数

第一组: read()和write()

第二组: recv()和send()

ssize_t recv(int socket, void *buffer, size_t length, int flags); 接收网络中的信息

返回值: >0 接收到字节数

==0 对方断开连接(重点, 单独讲解)

<0 失败

参数: 前面三个跟read意思一样

flags --》默认设置为0

ssize_t send(int socket, const void *buffer, size_t length, int flags); 发送信息到网络中

返回值: 成功 返回发送的字节数

失败 <0

参数: 前面三个跟write意思一样

flags --》默认设置为0

(9)关闭套接字

int close(int fd);

3.出现的一些问题

第一个问题: 客户端或者服务器强行退出, 导致read/recv不阻塞了

原因: 任意一方强行退出了(断开连接), read/recv由于找不到对方了,就不阻塞并且返回0(前面学习文件IO的时候, read读取文件, 读取完毕返回0)

应用: 写代码的时候利用read/recv的返回值判断对方是否还在线

第二个问题: 绑定失败!

: Address already in use //地址仍然在使用(误导你)

原因: linux系统中, 如果你的tcp/udp通信代码如果退出了, 绑定的端口号不会随着程序的退出立马释放(会间隔半分钟左右才会释放),你再次运行程序就会有绑定失败的提示

解决方法:

方法一: 耐心等待半分钟左右, 再次运行程序(不靠谱)

方法二: 换个端口号即可, 建议写成主函数传参

方法三：使用setsockopt()函数设置取消端口号绑定限制

4.tcp的基本特征和使用场合

(1)特征

- 有连接：通信双方需要事先连接成功，方可传输数据
- 有确认：一方收到对端的任何数据，都会给另一方发回执确认
- 保证数据有序、不重复、丢失会重发
- 如果网络拥堵，会自动调节发送量

(2)使用场合

- 可靠性高，传输质量要求较高，不能丢失数据
- 用户登录、账户管理等相关的功能

四.设置套接字的属性

```
int setsockopt(int socket, int level, int option_name, const void *option_value, socklen_t option_len);
```

参数：socket --》套接字文件描述符
level --》属性层次， SOL_SOCKET
option_name --》宏定义，表示你想设置套接字的哪种属性

SO_REUSEADDR //取消端口绑定限制,setsockopt设置取消端口绑定限制，代码必须写在socket()和bind()之间

SO_BROADCAST //允许udp广播信息

层级 (level)	选项名 (optname)	数据类型 (optval)	核心作用
SOL_SOCKET (套接字通用选项)	SO_REUSEADDR	int	允许端口复用
	SO_REUSEPORT	int	允许多进程 / 线程绑定同一端口
	SO_KEEPALIVE	int	开启 TCP 保活机制
	SO_SNDBUF	int	设置发送缓冲区大小
	SO_RCVBUF	int	设置接收缓冲区大小
	SO_LINGER	struct linger	关闭套接字时的延迟策略
	SO_BROADCAST	int	允许发送广播包

五.练习作业

- 完成服务器的代码，实现客户端可以给服务器发送信息
 - 验证服务器旧的套接字能否收发信息(不能)
 - 用多线程实现tcp双向通信，要求任意一方输入quit，客户端和服务要退出
 - 多个客户端连接同一个服务器
- 要求：服务器上面创建一个链表(什么链表都可以)，专门把连接成功的客户端信息(套接字,ip,端口号)保存到该链表
- 服务器 可以跟任意一个客户端一对一双向通信

5.用tcp实现客户端发送不同的请求给服务器，服务器做出不同的响应

比如： 客户端输入get 1.bmp 服务器就把1.bmp（800*480）这张图片通过网络发送给客户端，客户端保存
客户端输入get 1.txt 服务器就把1.txt这个文件发送给客户端，客户端保存